Process Management

MS SADIA EJAZ

Operating System Concept: Process(1)

- A process is basically a program in execution
- Each process is associated with its address space which contains code segment, data segment, stack segment etc.)
- A process can open several files.
- Multitasking operating system
  - Can run multiple processes concurrently. Example: UNIX, LINUX,WINDOWS
  - A process table is maintained by Operating System (OS) to keep track of these processes.

Processes and programs

The difference between a process and a program:

- Baking analogy:
  - Recipe = Program
  - Baker = Processor
  - Ingredients = data
  - Baking the cake = Process

Main OS Process-related Goals

- Interleave the execution of existing processes to maximize processor utilization
- Provide reasonable response times
- Allocate resources to processes
- Support inter-process communication (and synchronization) and user creation of processes

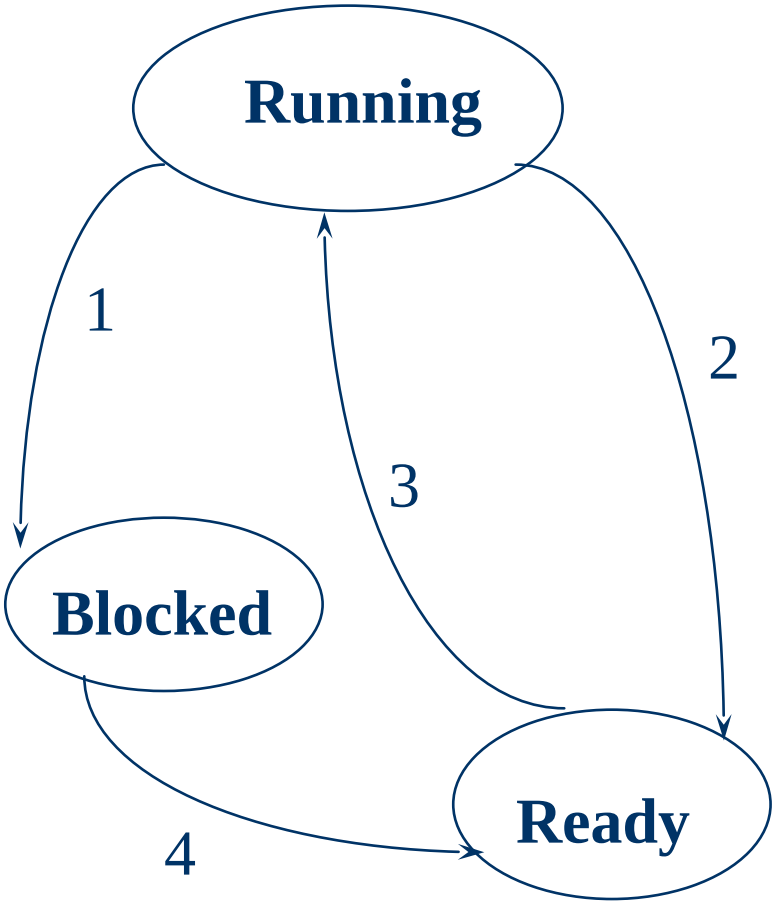
Process States

- A process is typically in one of the three states
  1. ***Running***: has the CPU
  2. ***Blocked***: waiting for I/O or another thread
  3. ***Ready to run***: on the ready list, waiting for the CPU

Process State Transitions

When do these transitions occur?

1. Process ***blocks*** for input or waits for an event
2. End of time-slice, or preemption
3. Scheduler switches back to this process
4. Input becomes available, event arrives

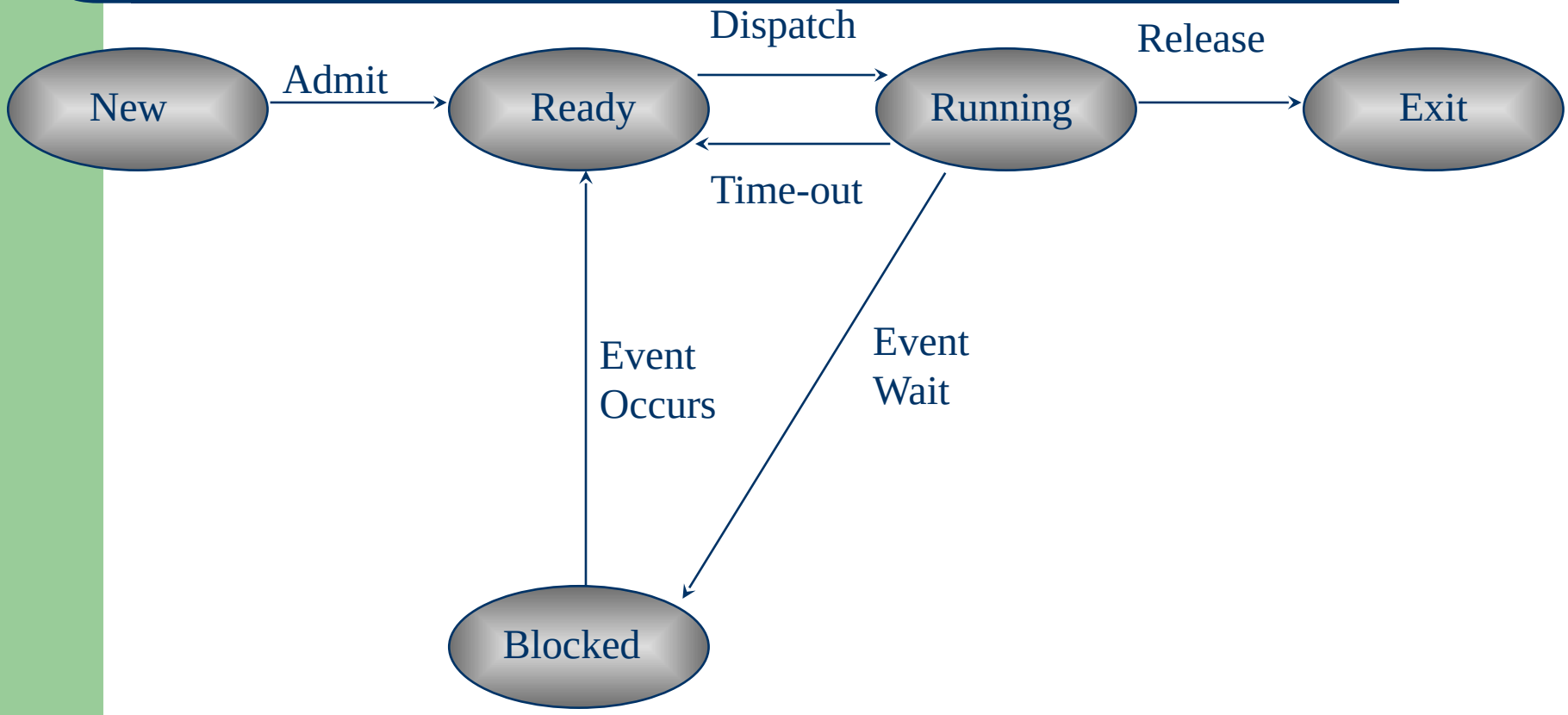


Process State(five state)

- As a process executes, it changes state
  - **new:** The process is being created.
  - **running:** Instructions are being executed.
  - **waiting:** The process is waiting for some event to occur.
  - **ready:** The process is waiting to be assigned to a process.
  - **terminated:** The process has finished execution.

Figure 3.2

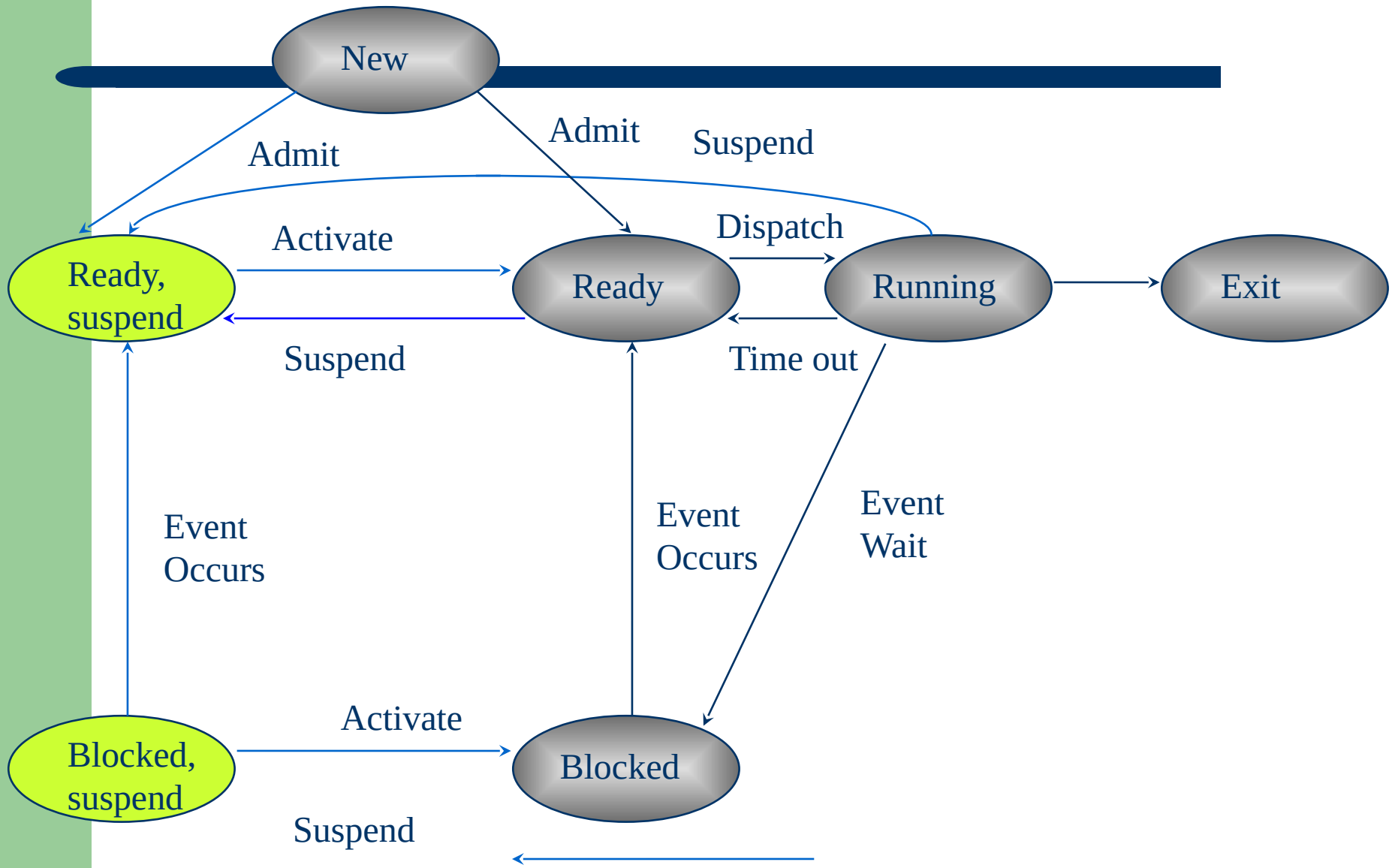
Five-State Process Model



Suspended Processes

- Processor is much faster than I/O so many processes could be waiting for I/O
- Swap some of these processes to disk to free up more memory
- Blocked state becomes blocked-suspended state when swapped to disk, ready becomes ready-suspended
- Two new states
  - Blocked-suspended
  - Ready-suspended

Process State Transition Diagram with Two Suspend States



Process Management Operations

- Process creation and termination
- Process scheduling and dispatching
- Process switching
- Process synchronization and support for inter-process communication

The OS maintains process data in the Process Control Blocks (PCB)

PCB

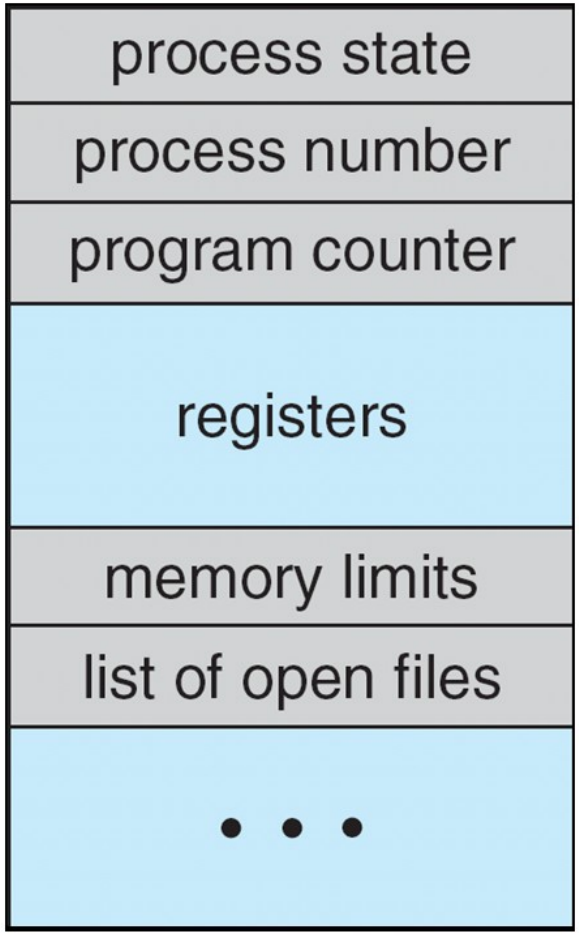
- Process Control Block (PCB)
- A Process Control Block is a data structure maintained by the Operating System for every process. The PCB is identified by an integer process ID (PID). A PCB keeps all the information needed to keep track of a process

Process Control Block (PCB)

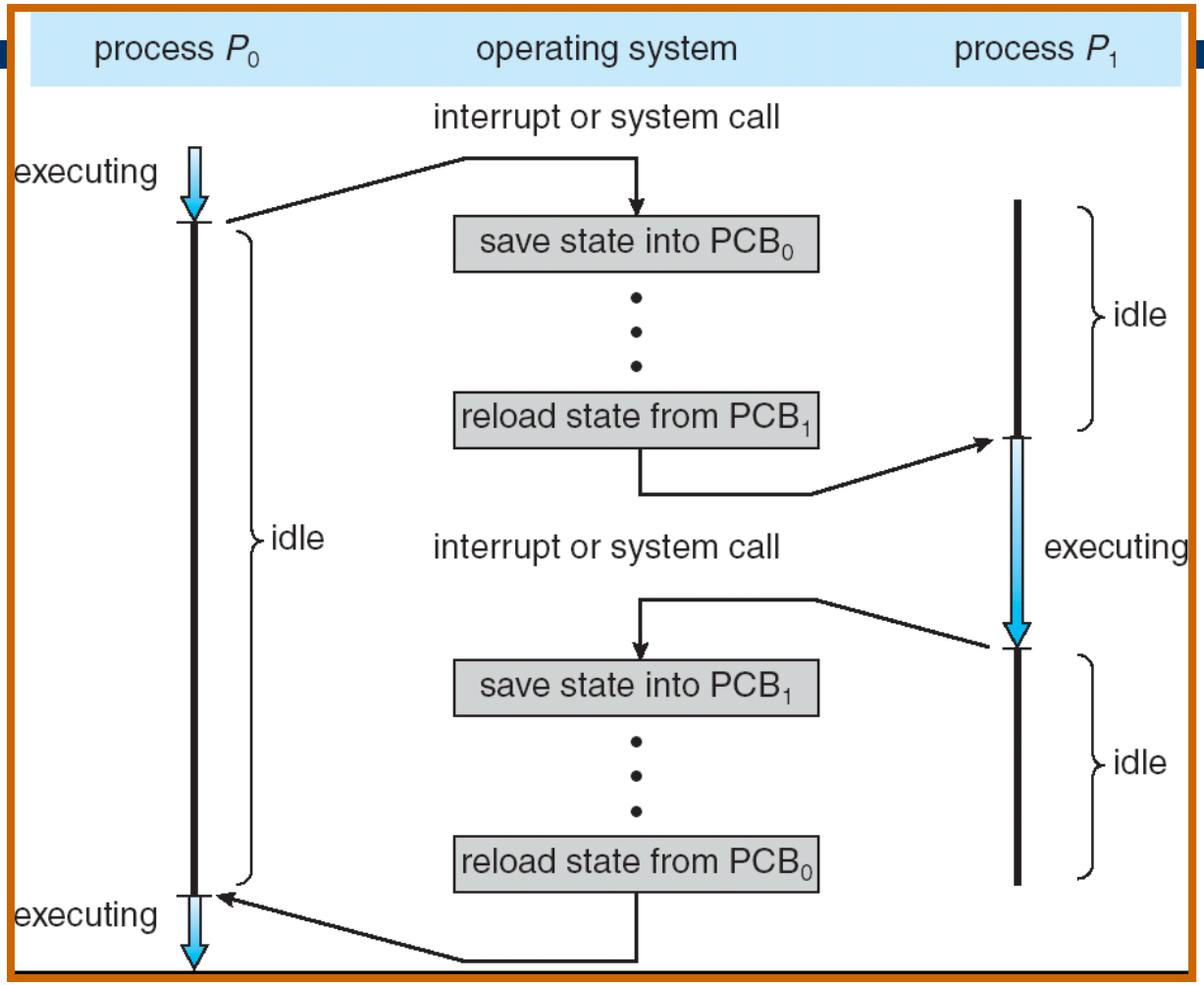
Information associated with each process

(also called **task control block**)

- Process state – running, waiting, etc
- Program counter – location of instruction to next execute
- CPU registers – contents of all process-centric registers
- CPU scheduling information- priorities, scheduling queue pointers
- Memory-management information – memory allocated to the process
- Accounting information – CPU used, clock time elapsed since start, time limits
- I/O status information – I/O devices allocated to process, list of open files



CPU Switch From Process to Process





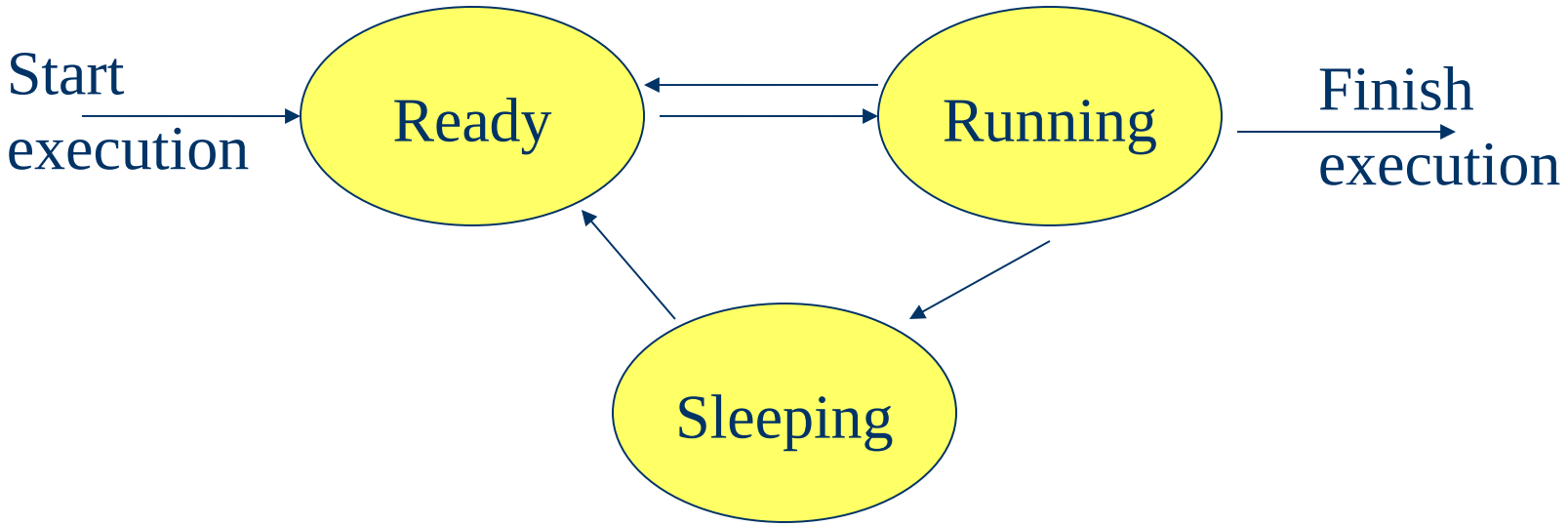
Case study: Linux

B.3 Process management

Linux Process Management

- Linux is a multitasking system
- Multiple programs can be executed at the same time
- Ultimately, a program needs to be executed by a CPU
- If there is only one CPU, how multiple programs can be executed at the same time?
  - ⇒ By time sharing
- That is, all programs are claimed to be executing. In fact, most of them are waiting for the CPU

- A program that is claimed to be executing is called a process
- For a multitasking system, a process has at least the following three states:





- Ready state
  - All processes that are ready to execute but without the CPU are at the ready state
  - If there is only 1 CPU in the system, all processes except one are at the ready state
- Running state
  - The process that actually possesses the CPU is at the running state
  - If there is only 1 CPU in the system, at most there is only one process is at the running state
- Sleeping state
  - The process that is waiting for other resources, e.g. I/O, is at the sleeping state

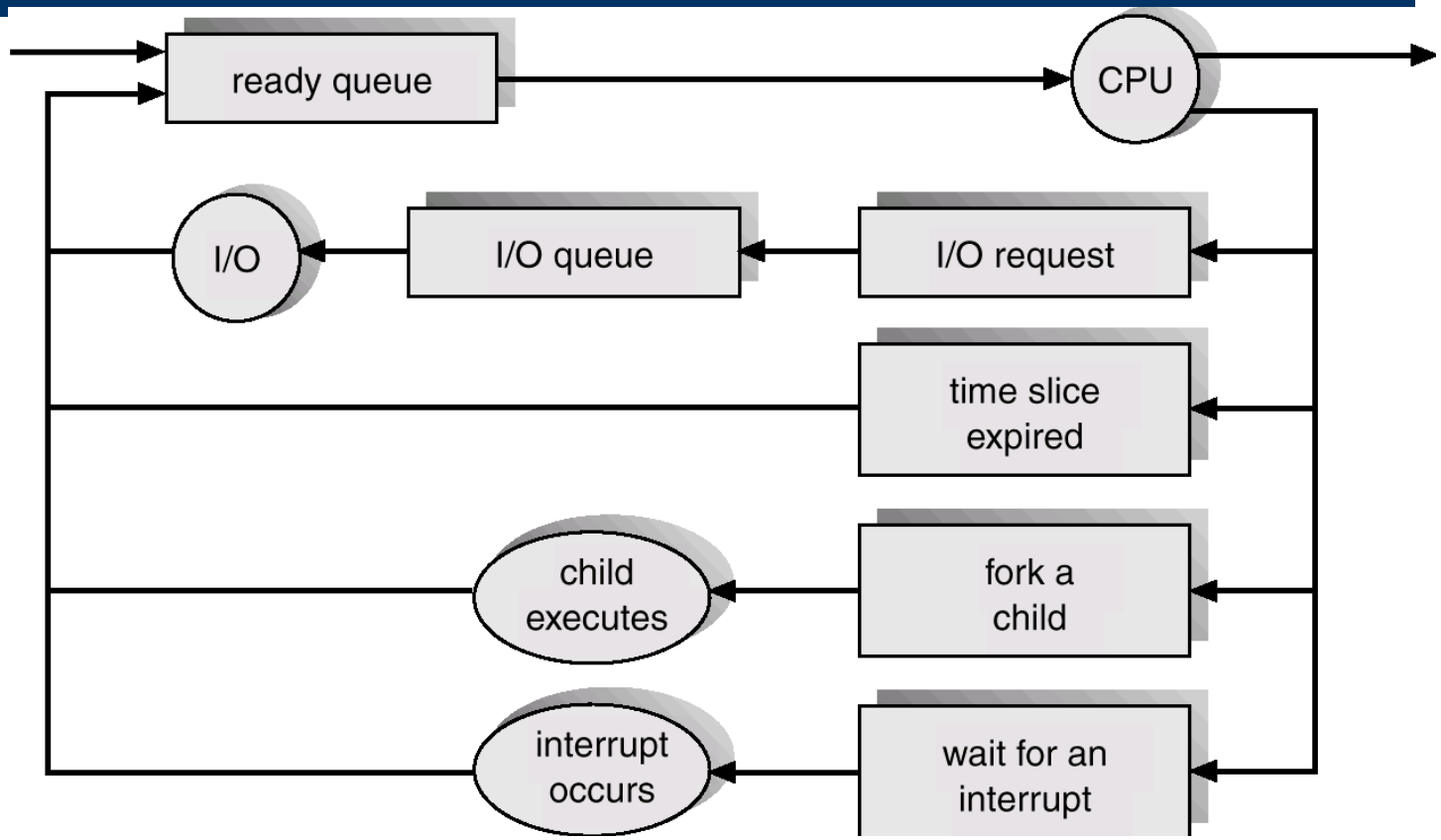


- Processes will alternatively get into the CPU one after the other (called the round robin scheme)
- A process will be “in” a CPU for a very short time (quantum)
  - For Linux, each quantum is about 100msec
- At the time that a process is selected to be “in” the CPU
  - It goes from ready state to running state
- After that, it will be swapped out
  - It goes from running state back to ready state
- Or it may due to the waiting of an I/O device, e.g. mouse
  - It goes from running state to sleeping state
- When obtaining the required resource
  - It goes from sleeping state to ready state

Process Scheduling Queues

- **Job queue** – set of all processes in the system.
- **Ready queue** – set of all processes residing in main memory, ready and waiting to execute.
- **Device queues –** set of processes waiting for an I/O device.
- Process migration between the various queues.

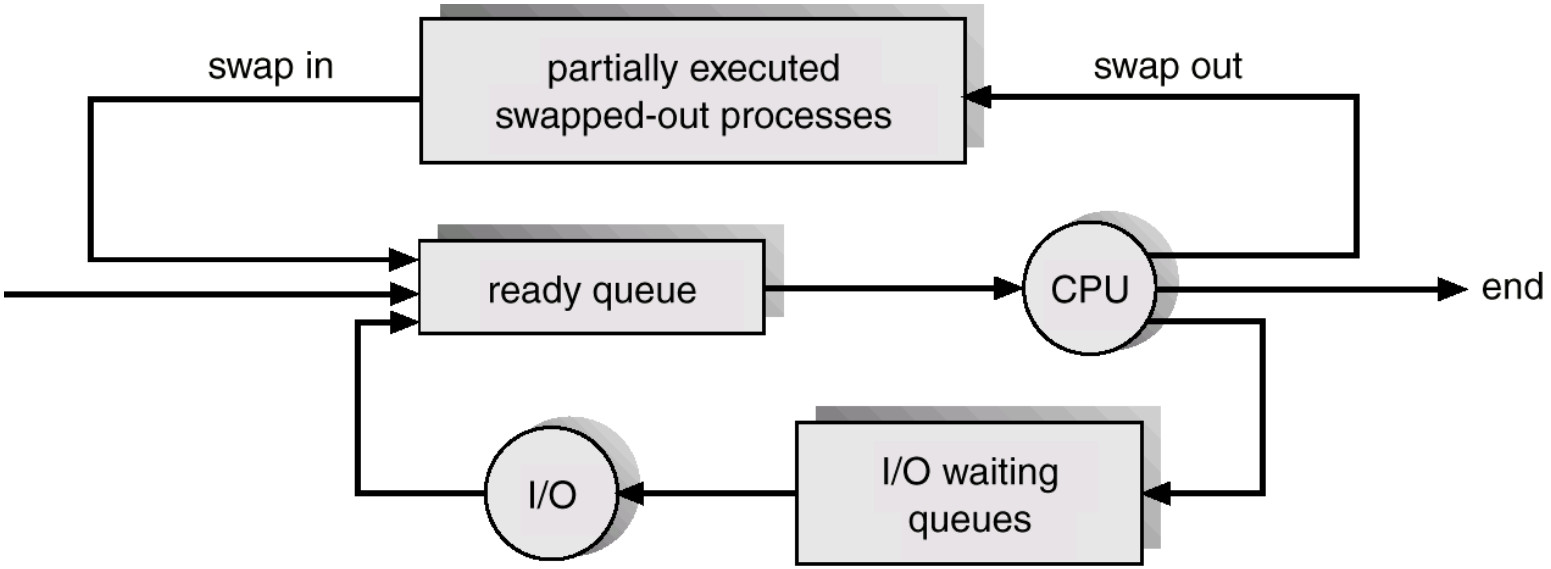
Representation of Process Scheduling



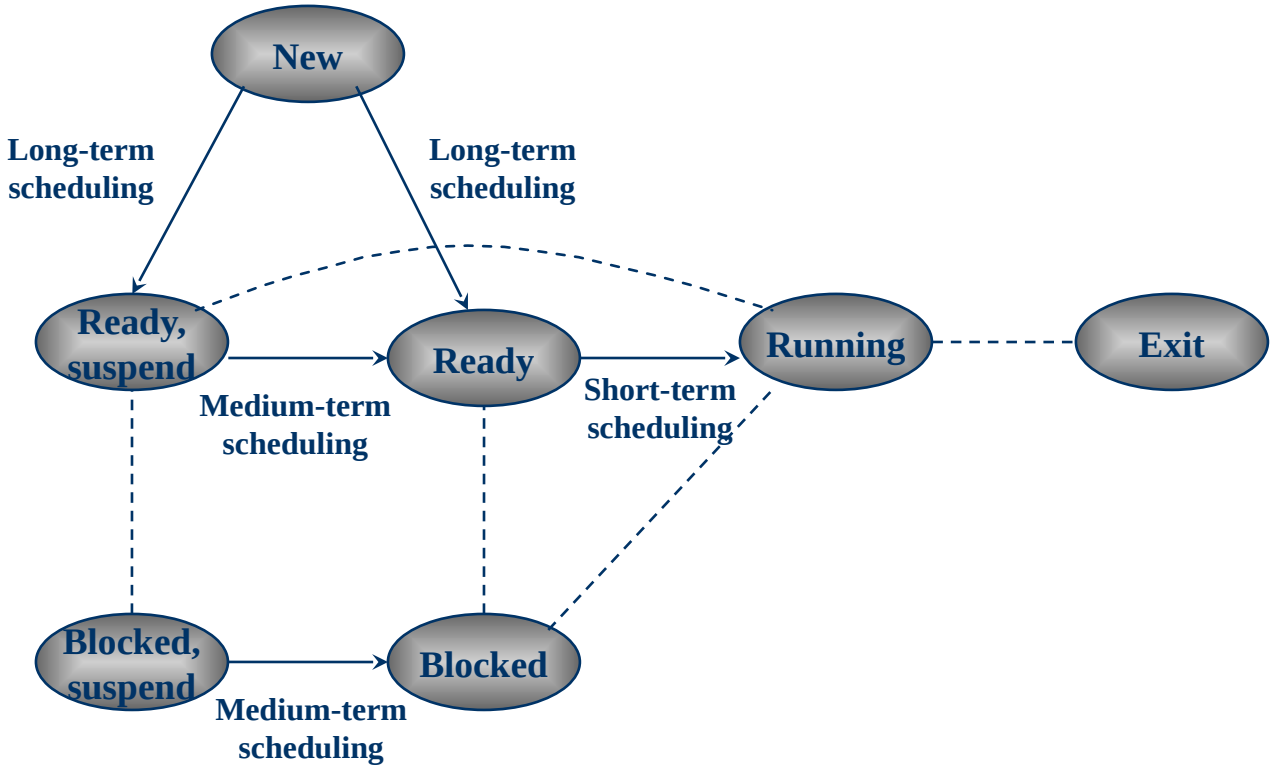
Schedulers

- **A process is placed in different scheduling queues throughput its life time**
- **OS must select these processes from these queues in some way for scheduling purpose**
- **The part of the OS related to this decision is called Scheduler**
  - **Long-term scheduler** (or job scheduler) – selects which processes should be brought into the ready queue.
  - **Short-term scheduler** (or CPU scheduler) – selects which process should be executed next and allocates CPU.

Addition of Medium Term Scheduling



Scheduling and Process State Transition



Schedulers (Cont.)

- Short-term scheduler is invoked very frequently (milliseconds) ⇒ (must be fast).
- Long-term scheduler is invoked very infrequently (seconds, minutes) ⇒ (may be slow).
- The long-term scheduler controls the degree of multiprogramming.
- Processes can be described as either:
  - I/O-bound process – spends more time doing I/O than computations, many short CPU bursts.
  - CPU-bound process – spends more time doing computations; few very long CPU bursts.

Process Scheduling

- The medium-term scheduler, or swapper, swaps processes in or out of memory
- The short-term scheduler, or dispatcher, allocates the CPU to a process that is loaded into main memory and ready to run.
- Typically, the dispatcher allocates the CPU for a fixed maximum amount of time.

Process Scheduling

- The main objective of the job scheduler is to provide the medium-term scheduler with an appropriate number of jobs.
- If too few jobs?
  - CPU may sit idle because all jobs are blocked
- If too many jobs?
  - The memory management system becomes overloaded, degrading system efficiency (thrashing).

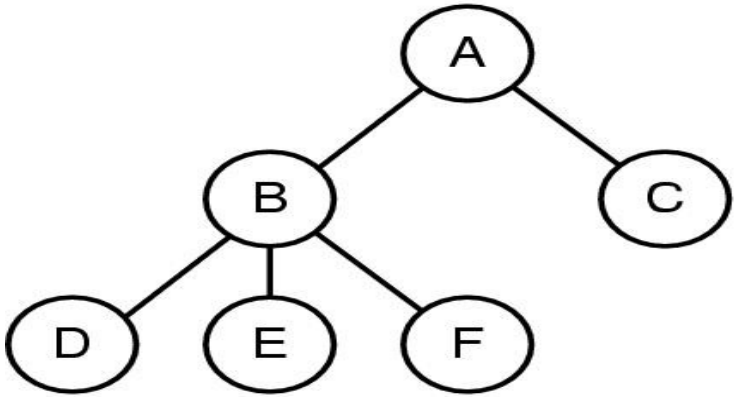
CPU Switch From Process to Process

- Switching a process requires
  - Saving the state of old process
  - Loading the saved state of the new process
- This is called ***Context Switch***
- Part of OS responsible for switching the processor among the processes is called ***Dispatcher***

Why is process context important?

- To run a process correctly, the process instructions must be executed within the process context!

IPC: Process



- A process can create one or more child processes
  - A created two child processes, B and C
  - B created three child processes, D, E, and F
- Inter Process Communication (IPC)
  - Sometimes it is needed to communicate between two processes.

Interprocess Communication

- Processes within a system may be ***independent*** or ***cooperating***
- Cooperating process can affect or be affected by other processes, including sharing data
- Reasons for cooperating processes:
  - Information sharing
  - Computation speedup
  - Modularity
  - Convenience
- Cooperating processes need **interprocess communication** (**IPC**)
- Two models of IPC
  - **Shared memory**
  - **Message passing**

Interprocess Communications

IPC methods include pipes and named pipes; message queueing ;semaphores; shared memory; and sockets.

Interprocess Communication – Shared Memory

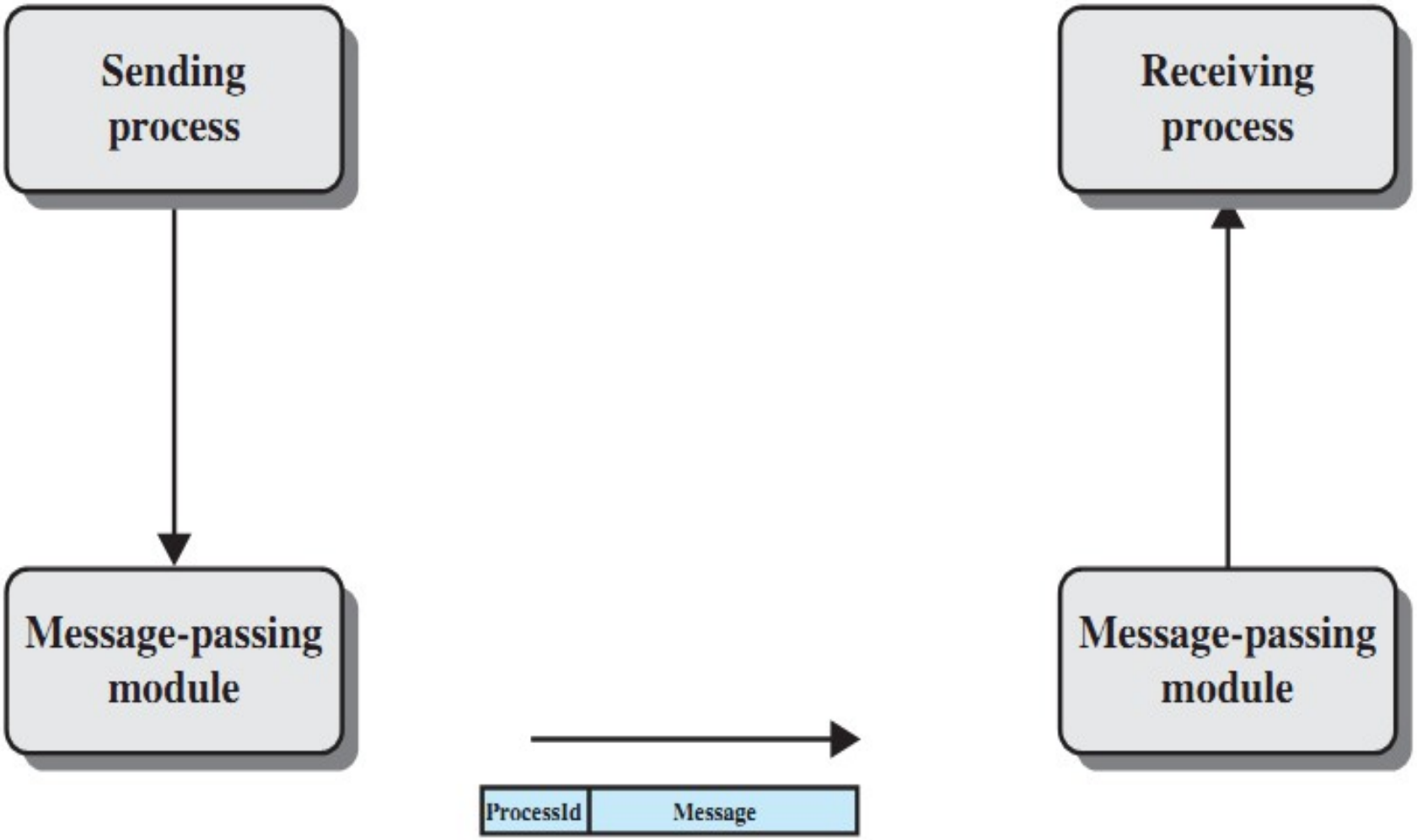
- An area of memory shared among the processes that wish to communicate
- The communication is under the control of the users processes not the operating system.
- Major issues is to provide mechanism that will allow the user processes to synchronize their actions when they access shared memory.
- Synchronization is discussed in great details in Chapter 5.

Inter-Process Communication (IPC)

- Mechanism for processes to communicate and to synchronize their actions.
- Message system – processes communicate with each other without resorting to shared variables.
- We have at least two primitives:
  - **send**(*destination*, *message*) or **send**(*message*)
  - **receive**(*source*, *message*) or **receive**(*message*)
- Message size is fixed or variable.



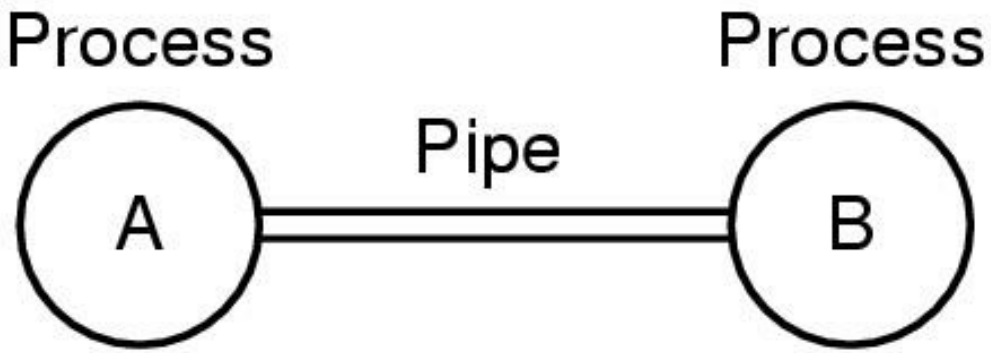
Basic Message-passing Primitives



pipes

- In computer programming, especially in UNIX operating systems, a pipe is a technique for passing information from one program process to another.
- Unlike other forms of interprocess communication (IPC), a pipe is one-way communication only. Basically, a pipe passes a parameter such as the output of one process to another process which accepts it as input.
- The system temporarily holds the piped information until it is read by the receiving process.

Operating System Concept: Process(3)

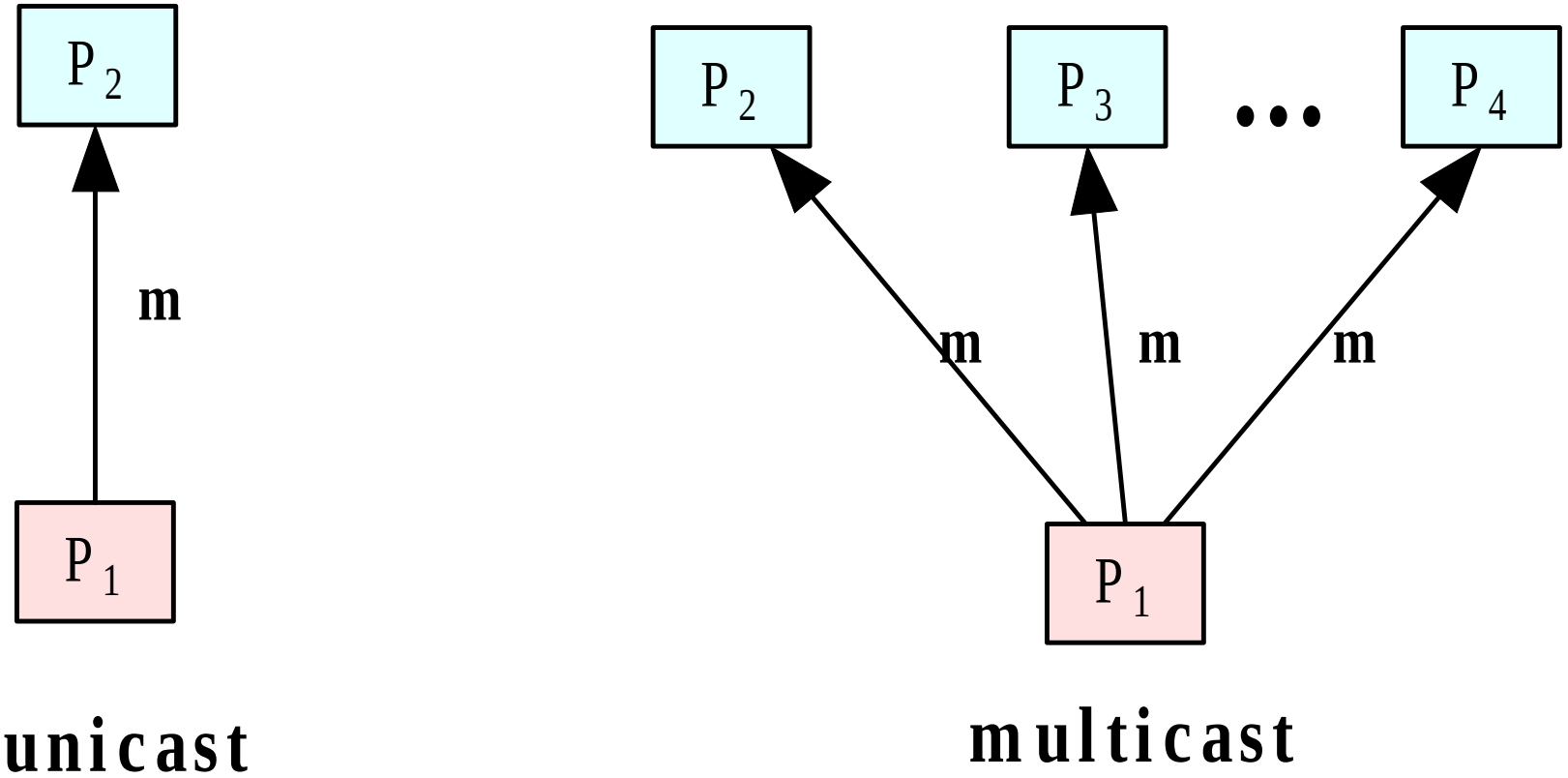


Two processes connected by a pipe

IPC – unicast and multicast

- When communication is from one process to a single other process, the IPC is said to be a ***unicast***.
- When communication is from one process to a group of processes, the IPC is said to be a ***multicast***.

Unicast vs. Multicast



Message Passing

- Message passing is a general method used for IPC:
  - for processes inside the same computer.
  - for processes in a networked/distributed system.
- In both cases, the process may or may not be blocked while sending a message or attempting to receive a message.



Synchronous vs. Asynchronous Communication

- **The IPC operations may provide the synchronization necessary using *blocking*. A blocking operation issued by a process will block further processing of the process until the operation is fulfilled.**
- **Alternatively, IPC operations may be asynchronous or *nonblocking*. An asynchronous operation issued by a process will not block further processing of the process. Instead, the process is free to proceed with its processing, and may optionally be notified by the system when the operation is fulfilled.**

Synchronization in message passing (1)

- Message passing may be blocking or non-blocking.
- **Blocking** is considered **synchronous**
  - **Blocking send** has the sender block until the message is received
  - **Blocking receive** has the receiver block until a message is available
- **Non-blocking** is considered **asynchronous**
  - **Non-blocking** send has the sender send the message and continue
  - **Non-blocking** receive has the receiver receive a valid message or null

Direct/Indirect Communication

- Direct communication:
  - when a specific process identifier is used for source/destination.
  - but it might be impossible to specify the source ahead of time (e.g., a print server).
- Indirect communication (more convenient):
  - messages are sent to a shared mailbox which consists of a queue of messages.
  - senders place messages in the mailbox, receivers pick them up.